How R2 works: rollback repair

DSA4213 verification-guided repair project · `chinatravel/agent/nesy_verifier/repair_strategies/r2.py`

Summary

R2 is **R0 with rollback**. Like R0, it asks the LLM for a full rewrite of the plan every time. Unlike R0, it throws away any rewrite that breaks a personal constraint the plan already satisfied, and it remembers which constraints those were. It also survives bad LLM replies instead of crashing.

On the R0-R3 spectrum, R2 sits on a different axis from R1 and R3. R1 and R3 restrict *what may change* (the scope); R2 keeps R0's full-plan scope and only changes *what may be kept* (the acceptance rule). An R2 vs R0 comparison therefore isolates the question: does protecting already-satisfied constraints help?

R2 runs inside the shared driver `run_repair_loop` in `shared_repair_loop.py`, the same loop every strategy uses.

Before the loop

1. `llm_modulo.py` generates the **initial plan**; this is not R2's work.
2. A new R2 instance is created for the query, with empty memory.
3. The loop verifies the initial plan (step 0). The result becomes the first **current plan** (the itinerary: a list of days with activities) and **current verification** (a dict whose `personal_constraints` field holds one True / False / None per constraint, plus `commonsense_errors`, `passed` and other fields).

One iteration

1. **Feedback.** R2 picks what to tell the LLM, using R0's feedback logic (`_build_feedback`) plus R2's memory notes. Personal phase: each failing personal constraint, and an "UNKNOWN; repair the plan so it can be evaluated" line for each None constraint. Commonsense phase: the commonsense errors only. Either phase: one line for every constraint in memory.
2. **Prompt.** ChinaTravel's `BACK_PROMPT`, the same template R0 uses: travel data, the query, the current plan as JSON, and the feedback. Sent as a single message with no chat history.
3. **Reply.** The LLM returns a complete itinerary, parsed the same way R0 parses it.
4. **Structural check (Case 4).** R2 keeps the current plan instead of the candidate if the reply is not valid JSON, is not a usable itinerary (no days, no activities, malformed days), or has the wrong number of days.
5. **Verification.** The loop checks the candidate against **all** personal constraints and commonsense rules.
6. **Decision.** The loop calls `accept()`, which compares the current `personal_constraints` with the candidate's, entry by entry. Any True -> False: reject, and the broken indices go into memory. Otherwise: accept.
7. **Update.** The loop logs the attempt. Only an accepted candidate replaces the current plan, so a rejected one leaves the previous plan in place for the next iteration.

Where the rollback lives

R2 decides; the shared loop restores. `accept()` in `r2.py` returns False on a regression:

```
regressed = _regressed_personal_indices(current_verification, candidate_verification)
for index in regressed:
    self._broken_constraints[index] = self._broken_constraints.get(index, 0) + 1
...
return not regressed
```

and `run_repair_loop` only replaces the current plan when the candidate is accepted:

```
if accepted:
    current_plan = deepcopy(result.repaired_plan)
    current_verification = candidate_verification
```

On a rejection nothing is overwritten, so the next `repair()` call receives the previous plan. The rejected candidate is still logged, with `accepted=False` and `accepted_plan_after` set to the previous plan.

The four cases

Case	What R2 does	Plan after the attempt	How it is logged
1. Fixes target, breaks nothing	accept() -> True	Candidate	accepted=True
2. Breaks a passing personal constraint	accept() -> False; constraint added to memory	Previous plan	accepted=False, decision_reason = previously_satisfied_constraint_regressed, rolled_back_constraint_indices; the rejected candidate is still logged
3. Target still failing, breaks nothing	accept() -> True	Candidate	accepted=True
4. No usable candidate	repair() returns a copy of the current plan	Previous plan	accepted=True (same plan, nothing regressed), failure_reason = no_valid_repair_candidate, candidate_error

- Case 2 applies whether or not the target was fixed. The only test is whether something that was passing broke.
- Accepted (not rollbacks): a constraint going from True to None, and new commonsense errors.
- Every attempt's metadata includes `target_constraint_indices`, `editable_scope: "plan"` and `protected_constraint_indices` (the constraints currently passing).
- To count rollbacks, filter on `decision_reason` (Case 2) or `failure_reason` (Case 4), not on `accepted`: Case 4 attempts are logged as accepted.

Memory of broken constraints

- **Stored as:** `self._broken_constraints`, a dict mapping each constraint index to how many rejected rewrites broke it, e.g. `{2: 1, 4: 2}`.
- **Written:** only in `accept()`, when a candidate is rejected.
- **Read:** in every later prompt, in both phases. Each remembered constraint gets a line:

> *A previous rewrite was rejected and rolled back because it broke personal constraint 2 (1 time(s)), which the current plan satisfies; keep it satisfied: <constraint code>*

- **Lifetime:** one query (a new R2 instance is created per query). Each attempt is logged with `remembered_broken_constraint_indices`.

Limits of the memory

- It only knows about constraints after they break once. The first rewrite gets no warning about passing constraints.
- The note only asks the LLM to keep the constraint satisfied; it does not enforce anything. The enforcement is accept(): if the LLM ignores the note, the rewrite is rejected again and the count goes up.
- It only covers personal constraints. Commonsense errors are never stored.

The iteration schedule

The shared loop decides this, and it is the same for every strategy. The budget is `min(--refine_steps, 6)`.

- **Up to 5 personal iterations**, while any personal constraint is not passing. Rejected attempts and Case 4 attempts still use up an iteration.
- **Then 1 commonsense iteration**, once all personal constraints pass or the 5 personal attempts are used up. It is skipped if there are no commonsense errors. If it breaks a passing personal constraint it is rolled back, and the commonsense errors stay unfixed.
- **Early stop** as soon as the current plan fully passes.
- **After the commonsense iteration**, the loop can return to personal iterations if any are left and budget remains.

Example run (2 constraints, constraint 1 failing)

Step	Current plan before	Feedback	Candidate result	Decision	Memory after
1	P0 [F, T]	constraint 1 failed	[T, F]	Case 2: reject; keep P0	{2: 1}
2	P0 [F, T]	constraint 1 failed + "you broke 2 before, keep it"	invalid JSON	Case 4: keep P0	{2: 1}
3	P0 [F, T]	same as step 2	[F, T]	Case 3: accept -> P1	{2: 1}
4	P1 [F, T]	same	[T, T]	Case 1: accept -> P2	{2: 1}
5	P2 [T, T], commonsense errors	commonsense errors + memory note	no personal regression	accept -> P3	{2: 1}

How R2 differs from R0

	R0	R2
Scope	Whole plan	Whole plan (same)
Prompt and parsing	BACK_PROMPT, R0 parser	Same, plus memory notes
Acceptance	Always accepts	Rejects any personal True -> False regression (rollback)
Memory	None	Constraints broken by rejected rewrites, listed in later prompts
Bad reply (Case 4)	Raises; the query's repair stops	Keeps the current plan and continues
Iteration schedule	Shared loop: 5 personal + 1 commonsense	Same

Comparison caveat. Because R2 survives bad replies that would end an R0 run, part of any R0 vs R2 difference may come from this robustness rather than from rollback.

Tests

`tests/test_r2_rollback_repair.py` (offline, mocked LLM and verifier): registration; accept() rejects only True -> False; rollback plus memory through the shared loop (Cases 1 and 2); Case 3; Case 4 for unparsable, wrong-day-count and no-activity replies; and R0 still raising on an unparsable reply. R2 has not yet been run against the real LLM or sandbox.